开源项目的死与新生！

腾讯程序员 腾讯技术工程 2025年9月19日 03:07 广东 164人



文末抽奖~

文末抽奖~

文末抽奖~

导语

「**生存还是毁灭？**」这对开源项目来说也是个问题。

> 「我曾见过那些你们人类不会置信的事。我曾目睹战舰在猎户座的边缘燃起烈火，我看过 C 射线在唐怀瑟之门边际的黑暗中明灭闪耀。所有这些瞬间，都会消逝在时间之中，一如眼泪消失在雨中。正是死期。」

许多开发者早期带着一腔热血，加入到开源社区，无私地贡献代码。 只是随着时间的流逝，人员变更、方向调整、技术迭代等种种因素接踵而至，许多开源项目最终逃脱不了的命运便是死亡。而我们所能做的便是尽可能地延缓它的死期。

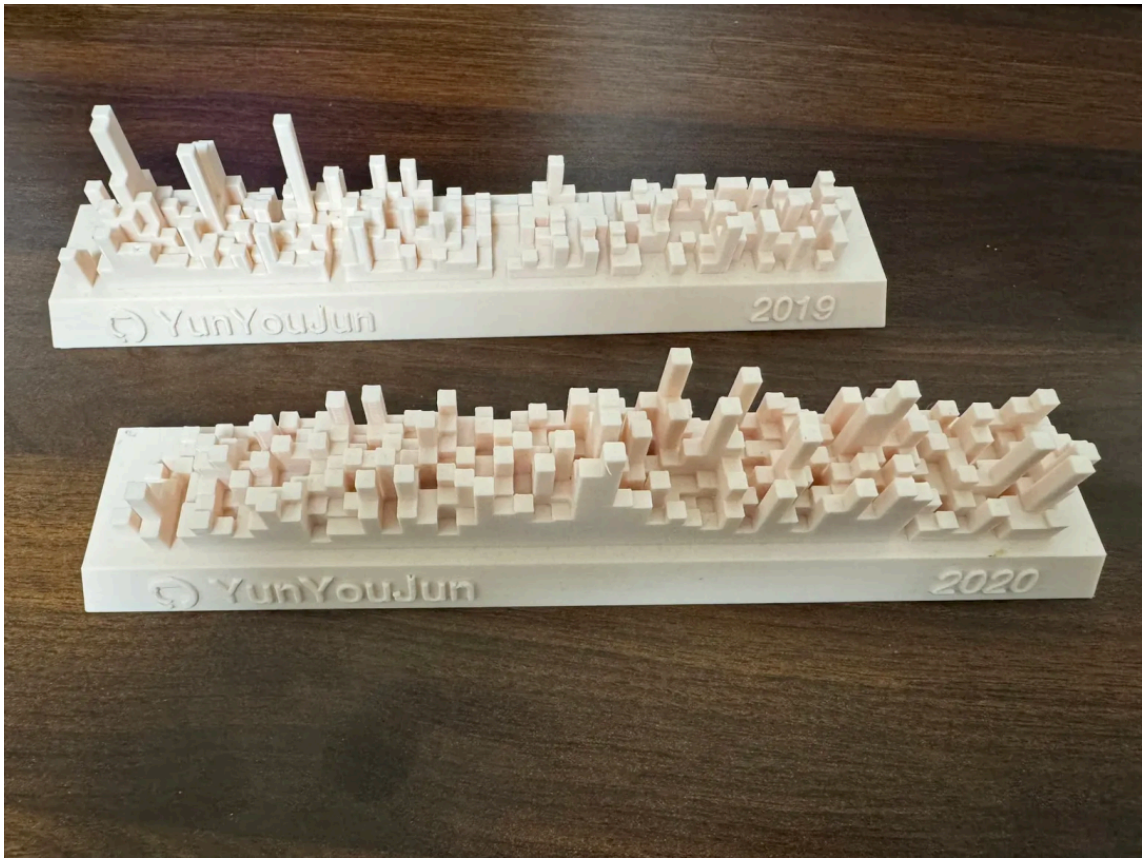
本文主要从作者自身视野出发，总结了一些关于项目开始与维护过程中，尽可能延长项目生命周期的个人经验。

自我介绍

大家好，我是云游君（**YunYouJun**），目前在腾讯 WXG 担任前端开发工程师。

我从学生时代开始积极参与开源社区内的各类项目，目前是 Element Plus (Vue UI 组件库) 的 Maintainer，也作为 Collaborator 为社区众多前端项目贡献过代码。此外也会开源一些个人~~整活~~项目和想法实验，在社区中总计已获得了 1w+ Star。

也伴随着历史的发展，见证了众多开源项目的死与新生。



关于 Element Plus

Element Plus 是一个基于 Vue3 中的 UI 组件库，目前也是前端社区中使用范围最为广泛的组件库之一。它脱胎于饿了么早期所开源的 Vue2 组件库 Element UI，并成为了 Vue3 版本的正式继任者。

它涵盖了大量日常开发中会使用到的 UI 组件，封装了相关常见的交互逻辑，并提供了周边相关的模版、插件等生态，旨在减少用户开发工作量并提供统一的设计。它的成长也并非一帆风顺，经历众多波折与维护者的更迭，逐渐完成了从公司主导的项目到社区开源项目、Vue2 到 Vue3 Break Changes 等的众多转变。

我有幸在早期参与，并得以见证它在社区中从 0 到 26k Star，周下载量达到近三十万的成长过程。

很高兴能有机会在这里和大家分享一些我个人在参与和运营开源项目过程中的一些体验心得，也希望能为大家参与司内/社区开源项目提供一些帮助。

关于开源

在参与开源项目之初，除了项目本身是否足够有趣/有用，我常常会先思忖这个项目是否会长久地活下去，再决定是否为其贡献代码。 倘若项目已进入弃用维护期，那么为其贡献代码的性价比便会大幅降低。 反之，项目早期的活跃贡献者总是更容易成为团队成员。而强者愈强，拥有更多活跃贡献者的项目，也更难进入死亡阶段。

因此，让社区的开源贡献者们相信项目会长久地活下去这一点，也非常重要。

开源的目的

开源项目的生命力与其根本用途息息相关。开源目的便是项目寿命的基因锁。

我们不妨先回顾一下开源的目的，**开源是为了什么**？

也许从更宏大一些的叙事角度来说，它是一种共享生产资料的形态，是共产主义的局部雏形，是推动技术生产力发展的动力。而从实际用户角度来说，它是公司/个人完成一个项目时，节约时间精力会依赖的一种方式。也有人觉得它使得程序员的一些经验没那么值钱了，但我觉得也正是信息资源的共享，使得我们减少了许多重复劳作，相关领域发展迅速、生活更加便捷。

小型初创公司乃至学生，都可以搭建自己的服务器或网站。开源社区的代码与经验分享，为 AI 的训练提供了庞大助力。如今借助 AI，几乎无开发经验的普通用户都可以快速创建一个网站或解决一些常见开发问题。我很难想象，如果没有开源社区，我们现今编写一个网站究竟需要花费多少时间和学习成本。

那些项目究竟是因何而开源？

有着很多诸如 Richard Stallman、Linus Torvalds 的先驱， 他们可能因更崇高的理想和兴趣，并在时代的背景下写下了那些代码。也有些项目来自庞大的科技公司，是生产中的实际需求，并通过开源来获取社区影响力与标准话语权。还有些则来自开发者们自身的需求，和学习过程中的实践。最终如繁星一般组成了丰富多彩的开源 Universe。

如上所述，开源的初衷各不相同。

但如 Linux、Chromium、Docker 这些已被众多开发者使用，并拥有一定标准话语权的项目，几乎很难死亡。与此同时，参与此类开源项目的前置经验也要求更为丰富。

对于我们普通开发者来说，又该如何开始？

我觉得 antfu 的一篇「关于 **Yak Shaving**」的文章写的很棒。

Yak Shaving 的字面意思是为剪牦牛毛，而引申出来的意思是，当你在进行一个工作时，发现另一个工作还没有完成，你便先去解决那个工作，在进行那个工作时，你又发现另一个工作…如此往复，让你偏离了原本本该完成的工作，最终却也什么都没有完成。

但反之，在这过程中，我们可以更好地发现现有项目的痛点，而着手改进。我想这也是初入开源最好的源动力。

对于自身有用的项目，是维护下去的源动力。也是个人项目活下去的关键。

什么是好的开源项目，什么又是「坏」的？

因此而诞生的开源项目数不胜数，在庞大的社区中，为了完成同一目的的项目可能有数种。它们可能各有千秋，我们不得不将其从浩如烟海的项目中筛选出来。正如磨刀不误砍柴工，初始正确的选择可以为后续节约很多开发重构成本。

我们可以通过星标数、项目/开发者活跃度、文档、用户 Issue 反馈、社区、体积/性能/学习成本、代码质量（乃至你是否认识开发者便于解决问题）等多个纬度来评判。好消息是我们现在可以借助 AI 来快速生成对比分析报告。不能武断地通过星标数评判项目质量，也没有绝对「好」的开源项目，只取决于是否适合你的需求。项目/作者活跃度也是一个不亚于星标的评判标准，再完美的项目总会有疏漏，这意味着你发现的问题反馈或PR后可以得到更快修复/合并。

但许多时候都如同「**小马过河**」，**任文档天花乱坠，亲手一试，方知深浅**。

不要重复造轮子，充分调研，参与贡献。

在我们自己开始一个项目前，我们应当总是调研出是否有现有项目符合需求，并分析其优劣。若不满足，则在文档中列出新的痛点设计，并在有信心维护好它再开始。 有时项目缺少的功能可能只是一个 PR 的事，这也会是我们参与社区的一个好的开始。

WindiCSS 诞生于 TailwindCSS 之后，且同样提供了原子化 CSS 的能力。两者看似有些重复，但 WindiCSS 从零开始的实现支持了按需使用功能，并探索了更多类如快捷方式、属性模式等特性。UnoCSS 紧随其后，它通过 preset 的方式重现了 WindiCSS 的功能，但更抽象化的设计使得更多的插件/预设成为可能。它也成功成为了 WindiCSS 的继任者。

这时这些项目的「重复」便是值得的。

勇于放手—承认项目的死亡并提供替代建议

在前端领域，重复类型的项目有很多，除了功能之外，视觉风格与底层设计可能是各项目诞生的初衷。也会有许多项目因为精力不足、底层框架变动、方向调整、人员变动等逐渐停止维护，这使得已使用了该项目的开发者群体苦不堪言。

虽然我们的主题是如何维护开源以延长它的生命周期，但各种因素之下也很难完全做到。

当项目已经积重难返，自己无力维护时，当自己/社区有了一个更好的新项目，且几乎可以平替该项目的所有功能时，承认旧项目的死亡也需要勇气。

提前提示项目即将进入 maintenance mode，并提供新的替代建议以供用户渐进迁移，而不是悄无声息地停止维护，让用户未来的某一日积重难返。

一鲸落，万物生。开源项目的死亡也常常推动着其他项目的**新生**。

我们肯定一些项目在时代背景下的贡献，但随着时间发展，项目设计已较为落后或是逐渐停止维护，但在此过程中仍然可能存在一些新项目选择基于此开发，较少的提示与几乎停滞的维护使得开发者们在未来的某一天仍然不得不将其重构。倘若是没有重构的机会，便不得不顶着众多陷阱继续向前。

在我此前的工作中曾使用过一种内部实现的包管理器，它的包管理机制设计部分与 yarn 平铺类似，但又并非完全相同，使得我们常常会遇到 yarn/pnpm 之外的其他边界问题，却又少了 pnpm 的安装速度。且默认不锁定依赖的策略，偶尔也会导致一些发布构建的边界问题。这类问题通常无法通过社区搜索自行解决，而不得不向上游层层传递，降低了开发效率。它虽曾做了一些司内的优化，但在 yarn[Q]/pnpm 已然流行的几年里，这些问题早已迎刃而解。

我想开源的意义之一也在于此，社区能够收集到更多的用户问题与边界 Case，从而使得项目更便于使用。是骡子是马，在社区中同台竞技，让用户自由做出选择，也能更好地促进项目本身的发展。

好项目，还是坏项目，用户说了算。

维护开源

随着时代发展、需求变更，许多开源项目总是不可避免地走向死亡，而维护便是不断改进项目并尽可能延长项目寿命的过程。

拥抱社区标准与开源，避免因不可抗因素而死亡

公司内常常有着众多因为不同业务需求而诞生的开源项目，但因为需求变更、开发人员的流动，许多项目在内部推广不久便又陷入危机。这其中不乏不通用的无效开源项目，但也有着好用的工具，让人惋惜。

有些项目因为强制推广而被引入，却又因种种原因停止维护而导致业务项目不得不重构迁移，会平白增添许多开发成本，而这份成本本身也很难被管理者看见，我想我们所力所能及的便是避免此类的项目的诞生与引入。

我此前的工作中，有一个老项目引入了集团的某个跨端框架，但由于其已几乎停止维护，部分特性无法与 React 新版对齐，我们后来不得不逐渐将其迁移重构。其他一些老项目使用到的状态管理库也遇到了相似的问题。此前我们维护的一些互动项目，基于社区 Pixi.JS 封装实现的库，尽管版本可能存在落后，但解决问题时，总是能够解决或在社区中寻找到类似的参考。而部分内部自行封装的库就没那么好运了，在重新维护时，常常面临着众多边界问题或原开发者的离职导致的停止维护，而不得不耗时耗力，另起炉灶。

再说好用的工具，维护项目的关键人员可能只有一两个人，核心开发者的离职，可能会直接导致相关项目的死亡。这部分工具有些会与内部基建/API 有所关联，有一套自定义的标准。但也有些足够通用的部分，并可通过适配器、插件的设计来应对各类业务需求。

对于此类项目，我认为维护其生命力的最好方法便是尽快推动对外开源，寻找更多的使用者与维护者。即便人员流失，原作者仍然可以参与维护相关项目的公共通用部分，并可以寻找更多的社区贡献者，减少维护成本。

Element Plus 是一个社区项目存活的典型代表。Element UI 最早来自饿了么的设计规范与 Vue 组件的封装，在早期收获了众多用户与贡献者。 在被第三方收购后，由于公司内部 React/Ant Design 的技术选型占据主流，Element UI 逐渐减少维护。原开发团队人员也已几乎全部离职，Element UI 陷入停止维护的边缘。 而此时正值 Vue2 升级 Vue3 的窗口期，较多的 Break Changes，使得社区少有可以使用的 Vue3 组件库。在内部的讨论决策后，Element Plus 得以从 Element UI 中诞生，并作为 Vue3 版本继续面向社区维护。此时的维护者大部份来自社区招募，而非公司内部开发人员。 并在过去几年内，维护与解决社区问题，一直保持着一到两周的发版频率，摆脱了因人员变动、方向调整等因素而死亡的命运。

仁慈的独裁者—避免分歧，敢于决策

> **终身仁慈独裁者**（英语：Benevolent Dictator For Life，缩写BDFL）是少数开源软件开发者所拥有的头衔。他们通常是某一项目的创始人，并在该项目社区出现争议时拥有最终的决定权。也是推进项目发展必不可少的角色。

在推进一个功能时，我们可以通过 Issue 创建 RFC（请求意见稿，Request for Comments）来获取社区的意见，并对新的 API 规范进行讨论，从而避免做出错误的决断。但与此同时，也可能存在始终无法达成一致意见的情况。

社区贡献者大多数因自身遇到了相关问题而参与贡献，他们热心且不索取经济回报，但反之他们并不总是有时间和精力参与项目的贡献。我们可以尽可能地招募热心贡献者，也需要相对稳定的维护者，同时也需要一位果断的决策者。

Node.JS 的社区繁荣而稳定，决策与贡献者相对较多且分散，这使得它的特性推进稳定但更为缓慢且不得不背负众多历史包袱。而 Bun 拥有一个绝对话语权的领导者，它的特性推进速度则十分显著。不过这并非模式的优劣之分，项目的不同时期，需要不同的角色，项目早期总是需要快速推进并解决大部份问题，而拥有庞大用户基数后，稳定则尤为重要。

社区项目时常会遇到一些 **Trade-Off**（权衡），我们需要在各类需求中做平衡，以满足大部份用户的需求，并向另一部份用户详细解释这样取舍的原因。Element Plus 的 Input 框拥有各类自定义的属性扩展，它默认的定宽和动宽在不同场景下会有不同的需求。我们总是无法满足所有用户的要求，这类细节的统计既耗费人力，也并不能收集到足够的反馈来代表某一方是绝大多数，而为了项目的推进，这时便需要决断其中一种方案。（对于拥有广泛基数的项目/规范来说，某个不经意的决定便可能造成长期影响并将错就错，直至新的 Break Change 出现与大部份用户迁移。）

> HTTP Referer，Referer的正确英语拼法是referrer。这是早期HTTP规范当中存在的拼写错误，后来为保持向下兼容将错就错。

用句中二的话来说，「**欲戴王冠，必承其重**」。我们有限的选择里无法保证某个决策一定是正确的，而相关的决策者也可能会在后续背负起错误决策的相关压力。

但只要项目本身还活跃着，那么这些问题都可以在未来的某一天得到改善。

对，就像生活一样，**只要活着，就有希望**。

不怕犯错，不断进步，不畏改变

总是花费大量时间讨论与重构设计，同样并不能促进项目的可持续发展。

在我自己的一个博客框架项目 Valaxy 初期开发中，我花费了大量时间与贡献者讨论插件系统对于某些边界情况的设计，并重构了多次，我们在某些默认行为上未达成一致，这使得相关功能 API 迟迟无法稳定。 但在我决定先按其中之一的路线推进时，有些问题却逐个迎刃而解了。Vite 作为众多前端框架的开发构建工具，其1.0 初期的插件设计与如今版本已大相径庭，2.0 期间，他们迅速对 API 重构，使得如今的插件系统能够满足各类需求。VitePress 在早期

并没有搬迁 VuePress 的所有功能（如本地搜索、插件系统），而这些在未来的发展中被一一补足。

维护项目并非一蹴而就，而是一个持久可持续的阶段。有时我花费大量时间适配的问题，也许依赖库在未来的某个版本变更了新的设计，相关问题也许便迎刃而解或有了更好的实现方式。

没有项目是完美的，设计和变更的同时，批评往往也会随之而来，勇于承认过去的设计缺陷并敢于推进当下的Trade-Off 与改变，是一个项目保持活力的关键。

将重心放在项目本身的主体功能与特色，快速推进，得到更多用户群体的反馈。 随着时间不断进步，也不畏未来某个时候的破坏变更，我想便是一个项目成长并保持活力而**不可或缺的历史**。

支持渐进式迁移

项目开发者不畏变更，积极推进各类特性，版本号常看常新，但此时用户的心中想必会五味杂陈。频繁的破坏变更，可能来自开发者初期决策的思考欠缺，这也会导致用户的不信任感。

正如众所周知的语义版本控制，在推进新的破坏变更时，我们应该尽可能在早期开发版本 `v0.x` 中完成。

当项目稳定后，则可以通过在 major 版本（如 v1.x）中，提前引入相关特性的切换开关或弃用提示，并提供过渡用法与渐进式迁移指引，从而得到用户的使用反馈。当然此时又存在包体积的 Trade-Off （我们可以通过子包、插件分包、特殊别名等各类方式来解决），但对于大部份场景来说这往往是值得的。

Vue2 到 Vue3 之间便存在 API 的巨大 Break Changes，而它更早地提供了 next 版本语法的体验，并后续在 2.7 版本中支持 Vue3 Composition API 语法的渐进式迁移。 在重构老项目的过程中也是如此，我们常常没有足够的时间精力将所有的内容切换至新的版本/语法，但可以通过添加一些特殊的适配代码，使得项目可以渐进式迁移。

维护社区

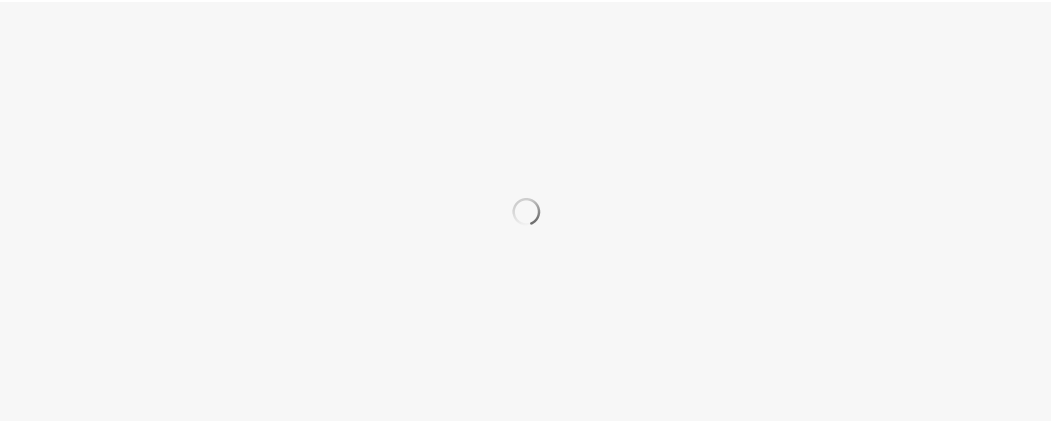
开源通常与社区一次并列提及，其重要性不言而喻。

以下是我认为维护一个好的开源社区氛围所可以做的：

- **加强与用户之间的联系**
  - **及时响应问题**，这一点至关重要。当用户通过 Issue 反馈相关问题，若能及时修复，则再好不过。即便暂无时间修复或无法修复，也可以留下一些相关解释说明。这会为提问者与发现到此处的其他用户留下好印象，而项目之间的印象是会口口相传的。在我们发现其他项目问题的解决方案时，也可以积极在评论区分享。
  - 更新与反馈的双向触达，**建立社交媒体或人设**。如果你是一位网络冲浪爱好者，在有余力的情况下可以为项目创建独立的社交帐号或使用自己的社交帐号公布一些项目的开发进展，使其更新进度可以触达更多用户。也可以通过群聊/频道/视频获得用户的直

接反馈，当然这并不是必须的，仍然应优先通过 Issue 管理相关问题，使得其进度也可以被其他人跟踪。

- **永远友善**
  - 作为个人社交媒体，我们可以拥有自己的情绪，但不应将其带入到项目本身。在维护一个大型开源项目时，我们有时也会遇到一些来自用户质疑与批评，即便心中颇有不忿，也应**永远保持友善的回复**。我们可以针对相关问题就事论事，解释各类决策的具体原因，毕竟**真诚才是必杀技**。在公开的 Issue/Discussion 中，相关内容也会被其他用户看到，而项目的印象也常会被与其开发者所关联。用户也可能是短暂冲动的，在维护 Element Plus 的过程中，我也曾遇到过一些语气恶劣的 Issue 与评论，但在承认错误或解释相关原因后，通常可以得到提问者的理解。
  - **Fix Typo？当然可以**。参与开源项目的人士中，也有不少学生群体。他们有时并未完全掌握社区的协作方式。也许他们只是在阅读文档，发现了一个错别字或文档样式而想要修改。但这同样是有价值的，给予引导并合并，这让开发者在未来更乐于贡献。也许未来某一次的 PR 便是一个重要的 Bug Fix。
- **在文档与代码留下设计思路、注释、引用来源**
  - 在文档中解释自己的设计方案，可以减少一些问题答疑，同时帮助用户更好地理解项目。而注释除了体现代码质量本身，也可以帮助热心的贡献者更快地理解项目并愿意为你的项目做出贡献。除此之外，这也便于日后的自己回忆思路、修复问题。
  - 提供与其他相似产品的优劣对比，为读者展示自身项目的优势/特点所在。
- **做好 Trade-Off，避免项目停滞**
  - 正如上文所说，项目初期与争议时刻，「仁慈的独裁者」是必不可少的。
  - 此外项目本身可能存在许多功能待完善，也需要分清优先级，推进项目本身。我的优先选择通常是高频的功能与让自己愉悦（开发者体验）的功能，零散时间则可以处理短时间可以解决的问题。
- **周期性发布但也不必吝啬版本号**
  - 对于小型项目的 Bug，可以不必吝啬版本号，在修复后即可发布。
  - 对于大型项目，一个 Bug 的修复可能也会与其他问题相互关联，即便存在测试保障，也无法短时间完成，则可以约定周期性发版计划（一到两周），固定时间发布。这对于下游的用户来说，也便于决定版本更新的时机。
- **增加开发贡献者曝光与归属感**
  - **文档曝光**，例如在生成文档时可以通过 Git 信息 生成展示对应内容的贡献者列表，这更利于促进贡献者的热情。
  - **社区归属感**，对于非常活跃的贡献者，可以考虑邀请其加入组织社区团队，增加团队归属感，它可以被展示在用户的身份中而被看到。



> 我想当开源项目基数足够多时，即便除开部分不通用而逐渐死亡的开源项目，仍会有不少项目存活下来。开源越多，留存下来的也总会更多。此外，邀请更多用户加入，社区组织的 Badge 展示，也是增加归属感与产生社区影响力的方式之一。

> 由于开源的形式，许多社区项目并不会有直接的经济回报。 不过鹅的 OTeam 会赠送社区礼物这一点我很喜欢！

像做产品一样做项目

每个项目其实也是一个产品，只是用户们大部份是开发者的身份。**长期主义**是产品长寿的关键。

- 不能为了开源而开源，做自己也会用到的开源项目。也就是吃自己的狗粮。希望别人愿意使用而参与的项目，自己必然更需要愿意使用。
  - 如果项目还没准备好，那便先取悦自己。（只是项目有时永远不会完美，也不会完全准备好，不断改进便总会更好。）
  - 做**有用/有趣/愿意长期投入维护**的项目。
- 开源的时机也与产品窗口期类似，过早开源与过晚开源，也可能有着各种结果。
  - 早期开源可以获取众多初期用户，Element 正是如此，但也因此在未来会存在着众多历史包袱。
  - 晚期开源对于用户来说，更容易被认为是重复的轮子，冷启动时间更长，但也不乏持续维护与更现代的设计而弯道超车的案例。譬如 Vite 之于 Webpack 等。
- 保障项目的质量，自动化 CI/CD、模版示例、Playground、文档，更好的「快速开始」则可以继续锦上添花。

开源的初心

我并非专业的全职开源玩家（不然也不会进厂打工了2333），也不会有机会在这里和大家分享。以上更多是我个人角度的评判，从我所喜欢的社区氛围与项目出发。

开源项目与工作的一大不同之处便在于你拥有更多的弹性选择（毕竟没有工资），你可以自由决定什么时候花多少时间/精力参与。 所以也并不需要为了参与而参与，我相信工作生活平

衡，心情舒畅才能想出好点子，写出好项目，做出好产品。

我参与开源项目的初心，既有觉得很酷的原因，也有希望能在简历上留下一笔通过面试的功利因素，而如今可能是更多地希望能够创作出一个能够帮助到他人并为别人留下回忆的产品项目。

> 我想多年以后，面对 GitHub，我将会回想起学生时代第一次为 Element UI 提交样式修复 PR 的那个下午。

福利时刻

既然聊到「死与新生」

就不得不提刘慈欣的《三体》了

不知道大家有看过么

今天就来抽10位粉丝

各送一套《三体》全集

参与抽奖方式：



抽奖奖品

抽 10 位粉丝赠送全套《三体》

长按识别小程序码

来抽奖助手，免费参与抽奖

点击卡片在公众号后台回复“三体”即可

周日下午6点准时开奖

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容

公众号



阅读 6120

留言

写留言

还没有留言，来写首评